TP1 Graphes

B-1) Voici la définition de la classe graphe avec ses deux méthodes.

```python
from PileFile import PileFile
import networkx as nx
import matplotlib.pyplot as plt

class graphe:

    def __init__(self, graphe):
        self.__graphe = graphe

    def dijkstra(self, mat_adj, sommet_depart, sommet_arrivee):
        observe=PileFile()
        visite=[]
        chemin=[]
        dejavu=[]
        cycle=False
        observe.enfiler(sommet_depart)
        distance={sommet_depart:0}
        while not observe.estVide():
            sommet=observe.defiler()
            if sommet in visite:
                cycle=True
            for voisin in mat_adj[sommet]:
                observe.enfiler(voisin)
                nouvelle_distance=distance[sommet]+mat_adj[sommet][voisin]
                if voisin not in distance or nouvelle_distance < distance[voisin]:
                    distance[voisin]=nouvelle_distance
                    dejavu.append(voisin)
                    visite.append(sommet)
        chemin.append(sommet_arrivee)
        for i in range(len(dejavu)-1, -1, -1):
            if chemin[-1]==dejavu[i]:
                chemin.append(visite[i])
        if sommet_depart not in chemin:
                chemin.append(sommet_depart)
        chemin.reverse()
        return chemin, distance[sommet_arrivee], cycle

    def dessiner(self):
        G = nx.DiGraph()
        weighted_edges = []
        for node_p, nodes_s in self.__graphe.items():
            for node_s, weight in nodes_s.items():
                weighted_edges.append((node_p, node_s, weight))
        G.add_weighted_edges_from(weighted_edges)
        pos = nx.spring_layout(G, seed=2)
        nx.draw_networkx(G, pos)
        labels = nx.get_edge_attributes(G, 'weight')
        nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
        plt.show()

if __name__ == "__main__":
    #liste d'afacence du graphe pour le test de l'algorithme
    adj = {'A':{'B':15,'C':4},'B':{'E':5},'C':{'E':11,'D':2},'D':{'E':3},'E':{}}
    adj_carte = {'Z':{'B':185, 'I':296},'B':{'I':245,'R':205},'I':{'R':200,'P':242},'R':{'C':179},'C':{'L':164,'P':332},'P':{'V':203,'M':168},'L':{'V':102},'V':{'M':213}, 'M':{}}
    #Main
    mon_graphe = graphe(adj)
    mon_graphe_carte = graphe(adj_carte)
    print(str(mon_graphe.dijkstra(adj, 'A', 'E')))
    print(str(mon_graphe_carte.dijkstra(adj_carte, 'Z', 'L')))
    mon_graphe.dessiner()
```

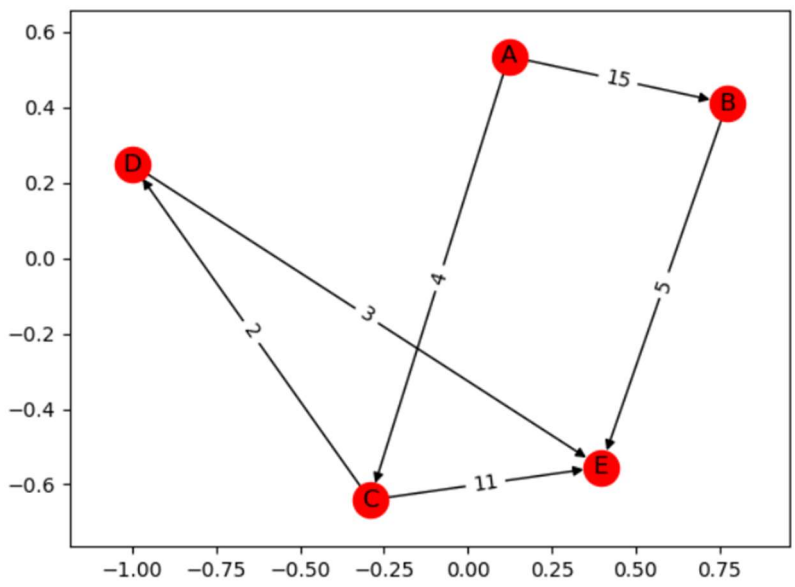
Voici ce qui s'affiche dans la console :

```
(['A', 'C', 'D', 'E'], 9, False)
```

B-2) Voici la méthode Dijkstra modifiée avec la présence d'une vérification de cycle :

```python
def dijkstra(self, mat_adj, sommet_depart, sommet_arrivee):
    observe=PileFile()
    visite=[]
    chemin=[]
    dejavu=[]
    cycle=False
    observe.enfiler(sommet_depart)
    distance={sommet_depart:0}
    while not observe.estVide():
        sommet=observe.defiler()
        if sommet in visite:
            cycle=True
        for voisin in mat_adj[sommet]:
            observe.enfiler(voisin)
            nouvelle_distance=distance[sommet]+mat_adj[sommet][voisin]
            if voisin not in distance or nouvelle_distance < distance[voisin]:
                distance[voisin]=nouvelle_distance
                dejavu.append(voisin)
                visite.append(sommet)
    chemin.append(sommet_arrivee)
    for i in range(len(dejavu)-1, -1, -1):
        if chemin[-1]==dejavu[i]:
            chemin.append(visite[i])
    if sommet_depart not in chemin:
            chemin.append(sommet_depart)
    chemin.reverse()
    return chemin, distance[sommet_arrivee], cycle
```

B-3) Voici le graphe pour la liste d'adjacence fournie. Le chemin le plus court est donc A-C-D-E



B-4) Voici la liste d'adjacence des successeurs de Z vers L :

```
adj_carte = {'Z':{'B':185, 'T':296},'B':{'T':245,'R':205},'T':{'R':200,'P':242},'R':{'C':179},'C':{'L':164,'P':332},'P':{'V':203,'M':168},'L':{'V':102},'V':{'M':213}, 'M':{}}
```

J'ai ensuite utilisé l'algorithme Dijkstra pour aller de Z vers L et voici ce qui s'est affiché dans la console :

```
(['Z', 'B', 'R', 'C', 'L'], 733, True)
```

C-1) On a cette fonction Python qui nous donne à partir d'une adresse les coordonnées GPS de cette adresse.

```
#fonction déterminant les coordonnées GPS à partir d'une adresse postale
def coord_GPS_addresse(adresse, ville):
    geo = Nominatim()
    coord = geo.geocode(adresse + ' ' + ville, timeout = 10)
    coord_GPS=(coord.latitude,coord.longitude)
    return(coord_GPS)
```

On fait ensuite l'appel de cette fonction :

```
# Programme principal
if __name__=='__main__':
    coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    print("Coordonnées GPS maison : ", coord_GPS_maison)
    print("Coordonnées GPS lycée : ", coord_GPS_lycee)
```

Voici ce qui est affiché dans la console :

```
Coordonnées GPS maison :  (48.8906761, 2.6075438)
Coordonnées GPS lycée :  (48.882752, 2.61227)
```

C-2) Voici le code utilisé :

```
def graphe(adresse_depart,Ville_D,adresse_arrivee,Ville_A,type_graph):
    origine_GPS=coord_GPS_addresse(adresse_depart,Ville_D)
    destination_GPS=coord_GPS_addresse(adresse_arrivee,Ville_A)
    point = ((origine_GPS[0]+destination_GPS[0])/2,(origine_GPS[1]+destination_GPS[1])/2)
    dist=distance_2pGPS(origine_GPS,destination_GPS)/1.5
    G = ox.graph_from_point(point, distance=dist, network_type=type_graph)
#    origine=ox.get_nearest_node(G, origine_GPS)
#    destination=ox.get_nearest_node(G, destination_GPS)
#    route = nx.shortest_path(G, origine, destination)
#    ox.plot_graph_route(G,route,fig_height=10, fig_width=10, show=True, use_geom=True, close=False, route_color='green' ,orig_dest_node_size=100)
    ox.plot_graph(G, fig_height=10, fig_width=10, show=True)
    return G
```

Et voici le graphe généré

C-3) Voici le code utilisé :

```
def graphe(adresse_depart,Ville_D,adresse_arrivee,Ville_A,type_graph):
    origine_GPS=coord_GPS_addresse(adresse_depart,Ville_D)
    destination_GPS=coord_GPS_addresse(adresse_arrivee,Ville_A)
    point = ((origine_GPS[0]+destination_GPS[0])/2,(origine_GPS[1]+destination_GPS[1])/2)
    dist=distance_2pGPS(origine_GPS,destination_GPS)/1.5
    G = ox.graph_from_point(point, distance=dist, network_type=type_graph)
    origine=ox.get_nearest_node(G, origine_GPS)
    destination=ox.get_nearest_node(G, destination_GPS)
    route = nx.shortest_path(G, origine, destination)
    ox.plot_graph_route(G,route,fig_height=10, fig_width=10, show=True, use_geom=True, close=False, route_color='green' ,orig_dest_node_size=100)
    return G, route
```

Voici l'appel de la fonction pour le trajet avec les transports en communs :

```
graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
```

Et voici le graphe généré :

Voici l'appel de la fonction pour le trajet à pied :

```
graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
```

Et voici le graphe généré :

A partir de cette question, j'utiliserai seulement le moyen de transport « drive » (voiture)

C-4) Voici l'appel de la fonction que j'ai réalisé pour récupérer la variable route pour le trajet en voiture :

```
# Programme principal
if __name__=='__main__':
    #coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    #coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    print("Route voiture : ", route_drive)
```

Et voici le résultat obtenu dans la console :

```
Route voiture :  [266608632, 266604653, 2304319952, 266595256, 267665162, 266594466, 266605101, 266605391, 8940931551, 8940931567, 8940931565, 8940931564, 8940931560, 267658481, 503850988, 503850917, 267655961
, 2635535955, 267228415, 267672928, 267228785]
```

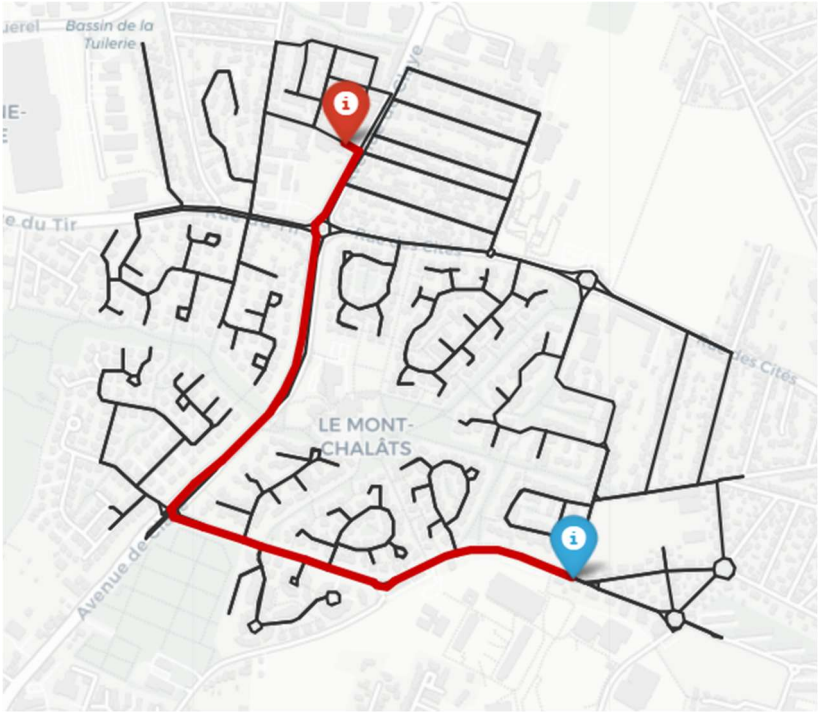
C-5) Voici le code exécuter (sans changement par rapport au fichier d'origine :

```
# génération d'une carte interactive itineraire.html avec tous les détails
def graphique_html(G,route):
    graph_map = ox.plot_graph_folium(G, popup_attribute='name',  edge_width=2)
    route_graph_map= ox.plot_route_folium(G, route, route_map=graph_map, popup_attribute='length')
    folium.Marker(location=(G.node[route[0]]['y'],G.node[route[0]]['x']), icon=folium.Icon(color='red')).add_to(route_graph_map)
    folium.Marker(location=(G.node[route[len(route)-1]]['y'],G.node[route[len(route)-1]]['x']), icon=folium.Icon(color='blue')).add_to(route_graph_map)
    filepath = 'itinéraire.html'
    route_graph_map.save(filepath)
    IFrame(filepath, width=600, height=500)
```

Appel de la fonction graphique_html(G, route) :

```
# Programme principal
if __name__=='__main__':
    #coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    #coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    #print("Route voiture : ", route_drive)
    graphique_html(G_drive, route_drive)
```

Page html obtenu :



C-6) Voici le programme principal utilisé :

```
# Programme principal
if __name__=='__main__':
    coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    #G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    #print("Route voiture : ", route_drive)
    #graphique_html(G_drive, route_drive)
    print("Orientation par rapport au Nord entre mon domicile et le lycée : ", orientation(coord_GPS_maison, coord_GPS_lycee))
```

Et voici le résultat dans la console :

```
Orientation par rapport au Nord entre mon domicile et le lycée :  201.41305785137013
```

Voici la capture d'écran que j'ai extrait de GIMP :

Voici la valeur de l'angle qui a été produite :

27.58°

On fait donc 27,58+180=207,58. L'orientation par rapport au nord est donc approximativement semblable.

C-7-1) Voici la fonction pour le calcul de la distance :

```
def calcul_distance(G, route):
    distance_totale=0.0
    for i in range(len(route)-1):
        sommet_courant = route[i]
        sommet_suivant = route[i+1]
        coord1 = (G.node[sommet_courant]['y'], G.node[sommet_courant]['x'])
        coord2 = (G.node[sommet_suivant]['y'], G.node[sommet_suivant]['x'])
        distance_segment = distance_2pGPS(coord1, coord2)
        distance_totale+=distance_segment
    return distance_totale
```

Et voici la fonction pour le calcul de la distance avec les arrêtes :

```
def calcul_distance_arrete(G, route):
    distance_totale = 0.0
    for i in range(len(route)-1):
        sommet_u = route[i]
        sommet_v = route[i+1]
        distance_segment = G[sommet_u][sommet_v][0]['length']
        distance_totale += distance_segment
    return distance_totale
```

C-7-2) Voici mon programme principal pour le calcul de la distance

```
# Programme principal
if __name__=='__main__':
    #coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    #coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    #print("Route voiture : ", route_drive)
    #graphique_html(G_drive, route_drive)
    #print("Orientation par rapport au Nord entre mon domicile et le lycée : ", orientation(coord_GPS_maison, coord_GPS_lycee))
    distance = calcul_distance(G_drive, route_drive)
    print(distance)
```

Et voici le résultat obtenu dans la console :

```
1663.4455820823655
```

Et voici pour le calcul avec les arrêtes :

```
def calcul_distance_arrete(G, route):
    distance_totale = 0.0
    for i in range(len(route)-1):
        sommet_u = route[i]
        sommet_v = route[i+1]
        distance_segment = G[sommet_u][sommet_v][0]['length']
        distance_totale += distance_segment
    return distance_totale
```

L'appel de la fonction :

```python
# Programme principal
if __name__=='__main__':
    #coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    #coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    #print("Route voiture : ", route_drive)
    #graphique_html(G_drive, route_drive)
    #print("Orientation par rapport au Nord entre mon domicile et le lycée : ", orientation(coord_GPS_maison, coord_GPS_lycee))
    #distance = calcul_distance(G_drive, route_drive)
    #print(distance)
    distance_arrete = calcul_distance_arrete(G_drive.adj, route_drive)
    print(distance_arrete)
```

Et voici le résultat dans la console :

```
1682.823
```

C-8-1) Voici la fonction codée pour obtenir le temps de trajet en seconde :

```python
def calcul_temps_trajet(distance, vitesse):
    vitesse_ms = vitesse/3.6
    return distance/vitesse ms
```

Voici l'appel de la fonction pour le trajet à 50km/h :

```python
# Programme principal
if __name__=='__main__':
    #coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    #coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    #print("Route voiture : ", route_drive)
    #graphique_html(G_drive, route_drive)
    #print("Orientation par rapport au Nord entre mon domicile et le lycée : ", orientation(coord_GPS_maison, coord_GPS_lycee))
    distance = calcul_distance(G_drive, route_drive)
    #print(distance)
    #distance_arrete = calcul_distance_arrete(G_drive.adj, route_drive)
    #print(distance_arrete)
    print(calcul_temps_trajet(distance, 50), 'secondes')
```

Et voici le résultat dans la console pour le trajet à 50km/h:

```
119.76808190993032 secondes
```

Environ 2min

Voici l'appel pour 20km/h :

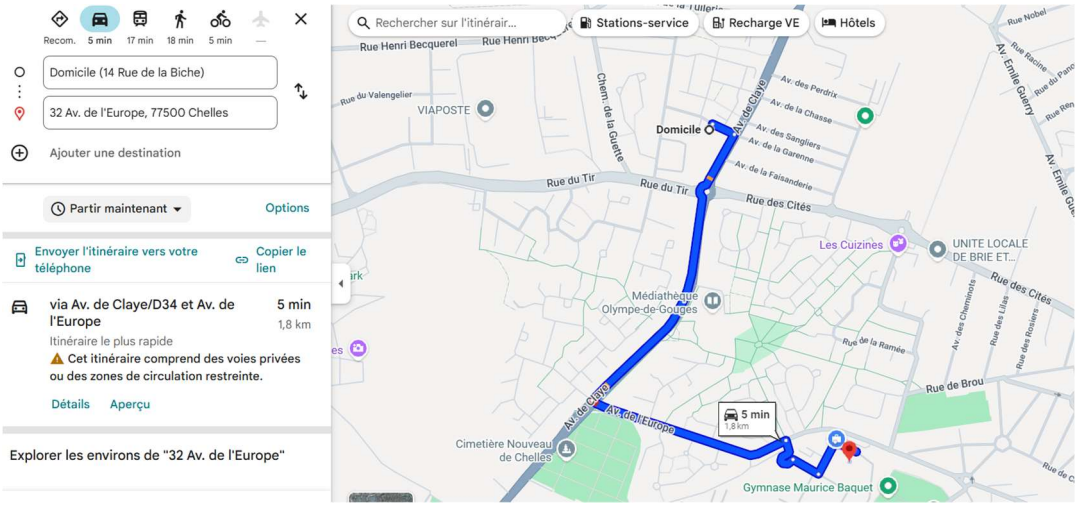
```python
# Programme principal
if __name__=='__main__':
    #coord_GPS_maison=coord_GPS_addresse("14 rue de la biche","Chelles")
    #coord_GPS_lycee=coord_GPS_addresse("32 avenue de l'europe","Chelles")
    #print("Coordonnées GPS maison : ", coord_GPS_maison)
    #print("Coordonnées GPS lycée : ", coord_GPS_lycee)
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive_service")
    #graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","walk")
    G_drive, route_drive = graphe("14 rue de la biche","Chelles","32 avenue de l'europe","Chelles","drive")
    #print("Route voiture : ", route_drive)
    #graphique_html(G_drive, route_drive)
    #print("Orientation par rapport au Nord entre mon domicile et le lycée : ", orientation(coord_GPS_maison, coord_GPS_lycee))
    distance = calcul_distance(G_drive, route_drive)
    #print(distance)
    #distance_arrete = calcul_distance_arrete(G_drive.adj, route_drive)
    #print(distance_arrete)
    #print(calcul_temps_trajet(distance, 50), 'secondes')
    print(calcul_temps_trajet(distance, 20))
```

Et voici le résultat dans la console :

```
299.4202047748258
```

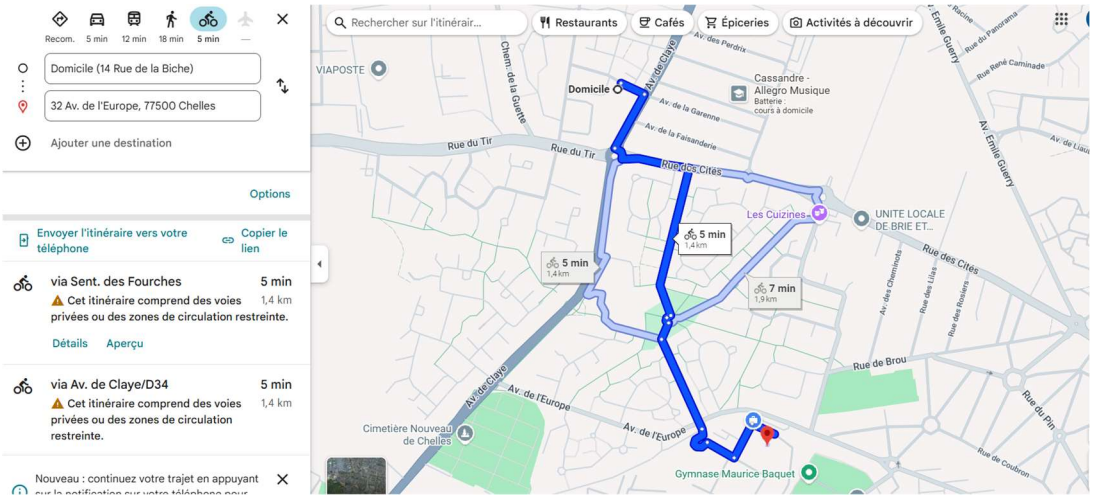
Environ 5min

C-8-2) Voici la copie d'écran de Google Maps en voiture



Les itinéraires proposés sont presque identiques mais le temps estimé est plus grand sur Google Maps. C'est normal car on a calculé le temps avec une vitesse constante de 50km/h alors qu'il y a des stops etc.

Voici la copie d'écran pour à vélo :



Le temps à vélo est plus fidèle car la vitesse fluctue moins.